Milliscale: Fast Commit on Low-Latency Object Storage

Jiatang Zhou
Simon Fraser University
jiatangz@sfu.ca

Kaisong Huang
University of Calgary
kaisong.huang@ucalgary.ca

Tianzheng Wang
Simon Fraser University
tzwang@sfu.ca

ABSTRACT

With millisecond-level latency and support for mutable objects, recent low-latency object storage services as represented by Amazon S3 Express One Zone have become an attractive option for OLTP engines to directly commit transactions and persist operational data with transparent strong consistency, high durability and high availability. But a naïve adoption can still lead to high commit latency due to idiosyncrasies of S3 Express One Zone and modern decentralized logging.

This paper presents Milliscale, a memory-optimized OLTP engine for low-latency object storage. Milliscale optimizes commit latency with new techniques that lowers commit delays and reduces the number of object access requests. Our evaluation using representative benchmarks shows that Milliscale delivers much lower commit latency than baselines while sustaining high throughput.

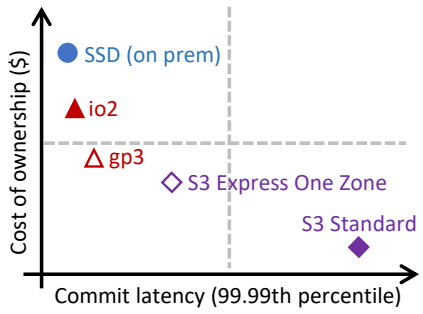

Figure 1: Commit latency vs. cost. Traditional object (S3 Standard) and block (local SSD, io2 and gp3) storage incur high commit latency and cost, respectively.

1 INTRODUCTION

Object storage services (e.g., Amazon S3, Google Cloud Storage and Azure Blob Storage) offer elastic storage, high availability (over 99.9% [5, 17]) and strong durability (11 nines [5]) at low cost [4]. They also allow very flexible accesses via HTTP requests. These features are highly desirable for database systems, as seen by the wide adoption of object storage for analytics [8, 12, 14, 15, 19, 28].

Although these features are also highly attractive for OLTP, using object storage as the main storage backend for OLTP engines to *directly persist write-ahead log as objects* is yet to become mainstream. A major reason is the high latency of traditional object storage. As Figure 1 shows, S3 Standard exhibits low cost but very high latency. As a comparison, block storage services (e.g., Amazon EBS gp3 and io2 [2]) allow significantly lower, ms-level commit latency, but come with much higher cost. Some also provide lower durability guarantees than S3 (e.g., two 9s in EBS gp3 vs. 11 nines of S3). Traditional object storage services also rarely support efficient updates (if any). Most existing OLTP services in the cloud thus use proprietary logging services [7, 13], high-end block storage (e.g., EBS io2) or object storage with SSD caches [8]. The downside is they increase system implementation complexity (e.g., by maintaining a separate internal service, or working around the immutability restrictions of S3 Standard [8]) and complicate deployment (e.g., by requiring specific compute instances with NVMe SSDs).

1.1 Logging on Low-Latency Object Storage

Recent low-latency object storage services as represented by Amazon S3 Express One Zone [3] offer (1) single-digit millisecond-level latency and (2) mutable objects that support append operations. These features are a significant improvement over S3 Standard's 100ms-level latency and immutable objects, leading to the interesting research question of *would it be feasible for an OLTP engine to transparently have high durability and fast commit by directly logging to an object storage service like S3 Express One Zone?*

To answer this question, we set out to adapt ERMIA [27], a memory-optimized OLTP engine, to flush log records to S3 Express One Zone. While S3 Express One Zone enables transparent high availability, high durability and flexible log accesses which can simplify functionality such as hot standby, we observe that transaction commit latency remains high. As shown in Figure 1, commit latency under S3 Express One Zone can be ~2× higher with cost approaching gp3's. We observe two key reasons below.

Excessive Object Append Requests. Modern DBMSs typically use decentralized logging [38, 41, 45] to avoid centralized logging bottlenecks [23]. The common approach is per-thread logging, which dedicates a private log buffer to each transaction worker thread. Log buffer size and the frequency of log buffer flushes directly determine commit latency. Using more parallel buffers can help ease the centralized logging bottleneck, and flushing more frequently/eagerly with smaller log buffers can help reduce commit latency. Both lead to a higher number of log flush (i.e., object append) requests, which in turn lead to more network roundtrips needed for flushing log records. Some modern memory-optimized DBMSs also tend to store full records in the log [37, 39] (rather than the potentially much smaller delta) to simplify implementation. This further increases log data volume, again increasing data transfer latency on the commit path. Moreover, cloud object storage services use pay-as-you-go [4] pricing models that charge by data volume, storage duration *and* the number of object access requests. As a result, higher number of requests also leads to higher cost.

Commit Dependencies. Under decentralized logging, to commit a transaction T, the OLTP engine must ensure T's own *and* depending log records which may be generated by other threads in their private log buffers [34] have been persisted. Compared to using a single log buffer, this naturally adds commit latency by

potentially requiring (1) flushing multiple log buffers and (2) dependency tracking [24, 33]. That is, a transaction T may read or overwrite the result of another transaction S recorded in another log. Committing T requires both threads' log buffers be flushed, adding commit latency. Worse, T may have to wait for the persistence of log buffers which it does not depend on, because its serialization order is later. Such false positive dependencies can lead to further higher commit latency.

1.2 Milliscale

This paper presents Milliscale, a memory-optimized OLTP engine that offers fast commit over low-latency, mutable object storage in the cloud. The core of Milliscale is a new decentralized logging protocol with optimizations that respectively reduce excessive log flush requests and unnecessary dependency delays.

We make two observations as we explore ways to reduce object append requests. (1) The frequency of log flush is directly affected by the speed the log buffer is filled, which in turn is determined by the amount of concurrent requests targeting each buffer. (2) On modern CPUs contention on a single log buffer by a small number of threads (e.g., 2–4) within a socket is negligible. Based on these observations, we depart from the common per-thread logging and propose *restricted decentralized logging*: Instead of assigning each thread a log buffer, we assign each group of (e.g., two) threads a proportionally larger (e.g., 2×) log buffer. As Section 4 elaborates, by carefully selecting log buffer size based on the performance characteristics of object storage, restricted decentralized logging reduces object append requests, yet without affecting commit latency.

On top of restricted decentralized logging, Milliscale reduces delays caused by false positive dependencies with a *lightweight record-level dependency tracking* approach. Instead of tracking at the page level [20, 38] or global durable commit timestamps [1], Each transaction T tracks the commit timestamp of its most recent direct predecessor P that generated the record read or updated by T. Once such P has committed, T can safely commit. This is done by keeping a transaction-local variable that is updated while reading/update records. The process is lightweight and only requires obtaining the commit timestamp on each version, which is usually already maintained by the underlying concurrency control protocols. With fine-grained record-level tracking, Milliscale avoids a large number of false positive dependencies that cause unnecessary delays.

Note that our goal is *not* to achieve the lowest possible commit latency and compete with NVMe SSD based solutions, but to show the potential for modern object storage of delivering satisfactory commit latency and throughput. We thus do not involve intermediary layers such as NVMe SSDs that are ephemeral and only available on certain compute instance types. Overall, the commit latency obtained by Milliscale is still higher than using NVMe SSDs. However, we believe this is a meaningful step towards cloud-native OLTP services with simpler system architectures but rich durability and availability features.

We built Milliscale on top of the aforementioned OLTP engine, ERMIA [27], whose logging subsystem was designed for block storage. Milliscale inherits ERMIA's memory-centric optimizations (e.g., indexing and concurrency control), but replaces ERMIA's logging subsystem with our new decentralized logging approach over low-latency, mutable object storage. Our evaluation using YCSB [11] microbenchmarks and TPC-C [36] on a 32-vCPU Amazon EC2 c6in.8xlarge instance and S3 Express One Zone shows that Milliscale delivers competitive throughput and much lower (up to 51.9%) tail latency when compared to baselines. In particular, Milliscale can bring the 99.99 percentile commit latency for write-intensive workloads of the unoptimized S3 Express One Zone to the level that is similar to gp3's, with the same elasticity and consistency guarantees as S3 Standard's.

Techniques in Milliscale are applicable to other object storage services. Restricted decentralized logging can be useful beyond logging that involve writing out buffers of data to storage (e.g., to reduce the number I/O requests in a busy system). One may further optimize the read path by adding SSD-based caches [8, 19]. We leave it as future work to explore these directions.

1.3 Contributions and Paper Organization

We make four contributions. ❶ We revisit the idea of OLTP over object storage and make the case for directly persisting write-ahead log on low-latency object storage. ❷ We propose a new decentralized logging approach that jointly optimizes throughput and commit latency. ❸ Based on the proposed techniques, we build Milliscale, the first memory-optimized OLTP engine that directly commits to low-latency object storage. ❹ We provide a comprehensive evaluation of Milliscale and baselines to validate our design decisions. Milliscale is open-source at https://github.com/sfu-dis/milliscale.

Next, Section 2 gives the necessary background on cloud storage and characterizes S3 Express One Zone to motivate our work. Sections 3–5 cover the design and evaluation of Milliscale. We discuss related work in Section 6 and summarize in Section 7.

2 BACKGROUND

In this section, we give the necessary background on low-latency object storage and logging to motivate our work.

2.1 Object Storage Basics

Object storage treats data items as "objects" which are identified by unique keys and accessed via HTTP requests, instead of file-based POSIX [21] APIs. An object is a byte container and multiple objects can be grouped together to be stored in a bucket. For example, Amazon S3 exposes `GetObject`/`PutObject` APIs that take a key as an input (along with other parameters, if any) and perform object read/write operations. Other APIs for operations such as delete and metadata queries are also provided. Requests can be issued from any compute client via data center networks or the internet, but most high-performance deployments use the former to lower latency, which we assume in this work.

Most cloud vendors offer object storage services with strong consistency and abundant bandwidth thanks to modern data center networking [15]. Although these services differ in their internal designs, they typically adopt S3 APIs which have become the de facto standard. We thus assume Amazon S3 in this paper and leave it as future work to explore other services.

Traditional object storage services (e.g., S3 Standard) are characterized as offering strong durability (e.g., 11 nines), strong consistency, high availability (over 99.9%) and low cost. The downside, however, is that they exhibit high access latency (100ms level) and objects are immutable (i.e., no update allowed once written). These have limited the use of traditional object storage mainly to read-dominant workloads such as analytics [15, 19]. Although some systems [8, 44] extensively use object storage for operational data, they need complex techniques to workaround S3 Standard's immutability and rely on aggressive caching using local NVMe SSDs that are only available on high-end compute instances.

2.2 S3 Express One Zone: S3 After 20 Years

Low-latency object storage services such as Amazon S3 Express One Zone improve traditional services with single-digit, millisecond-level latency and mutable objects via append operations.

Object Operations. S3 Express One Zone also exposes the HTTP-based APIs such as `GetObject`/`PutObject`. Append operations are issued using `PutObject` by specifying a write offset (`writeOffsetByte` in AWS S3 SDK) that is the end of the object. AWS's current implementation allows an object to be appended for up to 10,000 times [6]. Each append operation creates a "part" in the object and the maximum size per append (i.e., one part) is 5GB.

Performance. We issue S3 requests from an AWS EC2 instance to evaluate the latency of `GetObject` and `PutObject` (detailed setup in Section 5). Full-object writes behave similarly to append operations, so we omit their results and focus on append operations which are the most relevant to our use case (write-ahead logging).

As Figure 2 shows, S3 Express One Zone gives similar append latency (~8ms) for request sizes of 128–512KB. Beyond 512KB, append latency grows linearly. At 2MB request size, S3 Standard exhibits much higher latency of ~77ms, while the number for S3 Express One Zone is ~71% lower (~22ms). This means 512KB is the minimum request size to avoid excessive networking penalty. Since an object can accept at most 10,000 appends, the maximum object size is 4.88GB with 512KB request size. The DBMS must be able to segment the log into individual objects, which we describe later.

Retrieving data using `GetObject` is generally faster than writing for both S3 Standard and S3 Express One Zone. The latter exhibits ~5ms of latency for all request sizes of up to 256KB as shown in Figure 2. Reducing request size below this threshold yields no further benefit; increasing it to 1MB only marginally raises latency. Beyond 2MB request sizes, read latency starts to grow linearly.

These results indicate that the ideal request size for S3 Express One Zone is between 256KB to 1MB, potentially leading to an upper-bound latency of ~10ms for committing OLTP transactions, which is only 1/10th of what would be incurred using S3 Standard. We discuss how Milliscale leverages this observation later.

Availability and Durability. Compared to S3 Standard which replicates data across three AZs, S3 Express One Zone stores data in a single AZ to lower latency. The downside is lower availability compared to S3 Standard, leaving it to the DBMS that is built on top of S3 Express One Zone to mitigate. A straightforward solution is for the OLTP engine to replicate objects to additional S3 buckets in other AZs or regions.

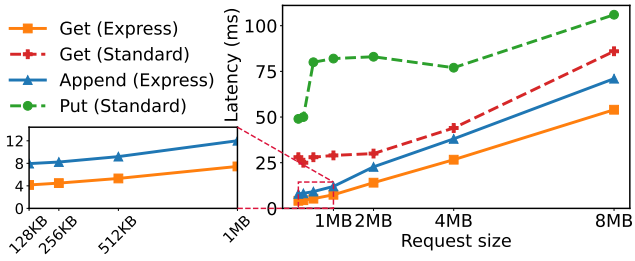

Figure 2: Object get, append, and put latency (ms) of S3 Standard vs. S3 Express One Zone under varying request sizes. S3 Express One Zone can offer single-digit ms latency at small request sizes (e.g., 512KB), making it a good fit for OLTP.

Pricing. S3 Express One Zone employs a pay-as-you-go pricing model to charge based on actual usage. This includes (1) storage cost which is charged based on the amount of and for how long the data is stored, (2) number of object access requests (e.g., pay per `PutObject`), and (3) bandwidth usage between the client (e.g., an EC2 instance) and S3. While storage cost is only related to data size and duration (similar to how other cloud storage), the other two items are closely related to the number of requests. As of early 2026, AWS charges $1.13 per million requests. On top of that, each GB of data uploaded to S3 Express One Zone costs $0.0032. The amount of data uploaded is in turn determined jointly by *the number of upload requests* × *request size*. For example, uploading 2GB of data by issuing 1 million append operations (i.e., 2MB or 0.002GB per request) will cost $1.13 + 1,000,000 requests × 0.002GB × $0.0032. The $1.13 is for issuing the 1 million requests alone, and the remaining is for uploading the 2GB of data. As a result, reducing the number of S3 requests (which is a major goal of our work) would also help lowering cost.

2.3 Why OLTP over S3 Express One Zone?

At first glance, S3 Express One Zone's pricing model is not friendly to OLTP which is characterized as having high write frequency. However, given the desirable features of S3 and drawbacks of other alternatives, we argue S3 Express One Zone can be a desirable choice for a few reasons.

First, services like S3 Express One Zone now provide the aforementioned strong consistency, high availability and strong durability, all *transparently*. This can greatly simplify the making of cloud-native database systems which have been mostly relying customized logging services maintained by teams internal to the DBMS vendor, such as the PLOG in Huawei Taurus [13] and XLOG in Microsoft Socrates [7]. Often times, without such proprietary services, block storage services like EBS are used, yet they do not always provide the same level of guarantees.

Second, S3 Express One Zone inherits the same flexible data access via HTTP requests from its standard counterpart. Objects are directly accessible by multiple compute nodes, simplifying hot standby setups where backup servers can directly read from the objects generated by the primary node. As a comparison, typical block services like EBS gp3 do not support accessing a volume at the same

time by multiple instances, or have limitations (e.g., io2's "multi-attach" feature allows multiple servers to mount the same volume, but does not allow them to access the data simultaneously under popular file systems like ext4 and XFS [9]). Without S3 Express One Zone, DBMS builders have to spend extra effort and employ additional hardware to deliver the aforementioned features. Such extra human capital cost is not typically taken into consideration when comparing service cost.

Finally, log space is usually bounded and reused with periodic checkpoints. This means the storage cost mentioned in Section 2.2 will stabilize instead of growing as data size grows, leaving the number of requests and sheer amount of data being transferred per roundtrip the only metrics to optimize for. Optimizing for these metrics is closely related to how modern write-ahead logging works, which we describe next.

2.4 Modern Decentralized Logging

Traditional write-ahead logging is centralized: all worker threads compete to insert into a single log buffer, which then gets flushed periodically to group commit transactions. Modern OLTP engines have largely moved away from this model to avoid contention on the centralized log buffer [24]. A common approach is decentralized, per-thread logging where each worker thread is allocated a private log buffer [37, 38, 41, 45], eliminating log buffer contention. Each log buffer is periodically flushed (e.g., upon group commit or time out) to stable storage, by appending to a file or object in S3.

Pipelined Group Commit. On top of decentralized logs, some systems use pipelined group commit [23] to improve throughput; we assume and design Milliscale with such optimization. With pipelined group commit, a transaction is detached from the worker thread once it "pre-committed," i.e., has finished processing all logic and inserted all log records to a log buffer. The transaction is then placed on a commit queue to wait for final commit. Meanwhile, the thread that was serving T can continue to serve another transaction. A background thread monitors the currently durable log sequence numbers (LSNs) and dequeues transactions and notify clients once their log records have been persisted.

Commit Dependencies. Pipelined group commit frees worker threads from being blocked by log flush I/O, but can form dependencies between transactions that require additional care upon commit. Consider a transaction T that updated record x and has just pre-committed, making T's results immediately visible. Any transaction S that accesses x will form a dependency on T: committing S requires log records generated by T be persisted. This was trivial in centralized logging as all log records are inserted to the only system-wide log buffer, flushing which will implicitly persist all depending log records [23]. Under decentralized logging, however, this often mandates flushing multiple log buffers. In the example, to commit S, the engine would require both log buffers serving two threads be flushed. Tracking exact dependencies (e.g., by maintaining a centralized dependency graph) can further incur performance bottlenecks [24]. Therefore, various prior approximations have been proposed. They typically involve maintaining global log sequence numbers (LSNs) and allow only transactions with commit LSN lower than the lowest durable LSN among all logs

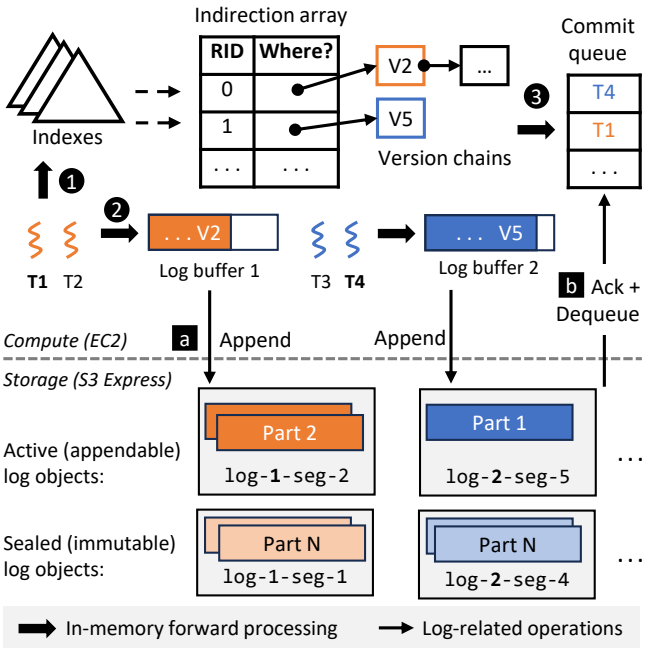

Figure 3: Milliscale Overview. Worker threads access records via indexes and version chains. The log is segmented into objects which are sealed once reaching append limitation. To reduce S3 requests while retaining high throughput and low commit latency, threads are grouped to share log buffers. Pre-committed transactions stay in a commit queue until their own and depending log records are persisted.

to commit [20, 26, 38]. We elaborate the issues of these approaches while introducing Milliscale's solution later.

3 DESIGN PRINCIPLES

The characteristics of S3 Express One Zone lead to the following design guidelines for Milliscale:

- **Frugal on Object Operations.** The size and number of requests of object operations (especially `PubObject` for logging) must be carefully calibrated and planned to reduce data transfer delays.
- **Short Dependency-Induced Delay.** The raw latency of S3 Express One Zone can be as low as single-digit millisecond. Milliscale should try to keep the end-to-end commit latency close to it by cutting unnecessary software-induced delays.
- **Low Latency while Sustaining High Throughput.** While our focus is to reduce (tail) commit latency, it is desirable to maintain high throughput as achieved by prior work.

4 MILLISCALE DESIGN

In this section, we first give an overview of Milliscale and then describe its design in detail.

4.1 Overview

Milliscale is an OLTP engine with a focus on reducing transaction commit latency over low-latency, mutable object storage. Like most

cloud-native DBMSs, Milliscale disaggregates compute and storage. Transaction requests are accepted on a compute node (e.g., an Amazon EC2 instance) which employs worker threads to access data and write out log records to object storage. We aim to optimize the commit path, so we focus on the logging subsystem and adopt representative memory-oriented optimizations from ERMIA [27] for multi-versioned concurrency control, indexing and data organization. This way, Milliscale maintains ACID properties while leveraging in-memory optimizations and object storage.

Compute Layer (Forward Processing). Records in Milliscale are identified by logical record IDs (RIDs). Versions of the same records are linked in a version chain in new-to-old order [40]. Each table is represented by an indirection array that maps each RID to the latest version of the record. Each version is stamped with by commit timestamp which is the commit sequence number (CSN) of the transaction that created it. CSNs are drawn from a central counter using the atomic fetch-and-add instruction [22] when the transaction entering pre-commit. In addition, each transaction is associated with a begin timestamp obtained by reading the global CSN counter upon creation or reading the first record. As shown in Figure 3(top), ❶ transactions traverse indexes and version chains to access data, and ❷ insert into log buffers as they modify the database. On top of decentralized logging, Milliscale allows limited sharing of log buffers among worker threads (but without introducing a bottleneck) and writing only the modified fields of each record. These designs collectively reduce S3 append requests.

Storage Layer (Commit Path). Following classic pipelined commit [23], ❸ after a transaction T finishes forward processing, the worker thread pre-commits T by placing it onto the commit queue. The thread then continues to handle the next request, while T waits for its own and depending log records to be persisted in S3.

Milliscale uses S3 Express One Zone as the main persistent home of data to benefit from S3's high durability, scalability and low-latency append operations. Importantly, Milliscale does not rely on intermediate levels such as NVMe SSDs on high-end compute nodes, allowing it to be deployed on low-cost compute instances. ⓐ Log buffers are written to S3 Express One Zone periodically using S3 append operations when they are full or reach a timeout. As shown in Figure 3(bottom), the log is segmented into objects, each of which consists of parts appended to it as a result of log buffer flushes. In S3 Express One Zone, Milliscale maintains an active object per log and each log buffer flush appends a part. For example, in Figure 3(bottom), log-1 is shared by $T1$ and $T2$, and currently has two parts in its active object. Meanwhile, $T1$ has inserted its new version $V2$ into Log buffer 1 and pre-committed. After Log buffer 1 has been filled or a timeout happens, it will be appended to the currently active object (log-1-seg-1) as the third part. The active object is sealed (immutable) once it reaches the append limit (10,000 times as mentioned in Section 2.2). For example, log-1 in Figure 3(bottom) has one sealed log segment (log-1-seg-1), whereas log-2 (shared by $T3$ and $T4$) has four sealed segments (log-2-seg-1 – log-2-seg-4). ⓑ Upon each successful append, we examine the commit queue to release transactions whose own and depending log records are all persisted.

Overall, commit latency is determined by for how long a transaction remains pre-committed, which is determined by (1) the size and frequency of log buffer flushes, and (2) dependency tracking delays.

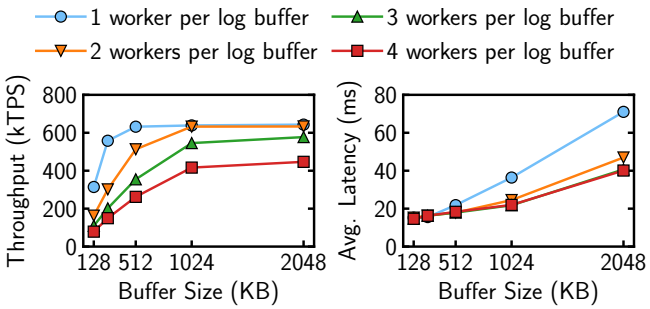

Figure 4: Throughput (left) and average commit latency (right) of TPC-C under different log buffer sizes and sharing ratios. S3 latency dominate with a 4:1 ratio, whereas a 2:1 ratio leads to the highest throughput and lowest latency.

In the rest of this section, we describe Milliscale's approaches to reducing both kinds of delays, while maintaining high throughput.

4.2 Restricted Decentralized Logging

Traditional decentralized logging dedicates each worker thread a log buffer to completely eliminate log buffer contention. However, it mandates more log buffer flushes. The same amount of work done by a group of parallel threads (generating the same amount of log data as using one log buffer) is now distributed to multiple log buffers, each of which needs to be persisted upon commit. Recent NVMe SSDs have further enabled such designs with high bandwidth and low latency [18]; it is often desirable to parallelize as much as possible with per-thread logs. On object storage, however, this is not always the case: using more log buffers increases both the number of S3 append requests and stragglers due to networking conditions, leading to lower performance.

Per-Group Logging. We observe the key culprit is the high number of log buffers in modern multicore systems, reducing which would naturally mitigate the problem. Therefore, instead of adopting traditional per-thread logging, Milliscale takes a middle ground to propose *restricted decentralized logging* which makes the degree of log decentralization adjustable. Restricted decentralized logging employs per-thread-group logging where worker threads are divided into groups, each of which is allocated a separate log buffer. Threads within a group will share and contend for the same log buffer (e.g., two threads per log in Figure 3). As Section 5 shows, however, such contention has negligible performance impact when concurrency (i.e., group size) is carefully controlled. That is, the log is decentralized across a set of thread groups, whose size is adjustable depending on the tolerable contention level on the given hardware. This allows us to reduce the total number of log buffers in the system, reducing the number of S3 appends and impact of stragglers on performance (i.e., improving tail latency).

Proportional Log Buffer and Thread Group Scaling. While the idea of sharing a log buffer per group of thread is straightforward, for it to be truly effective, one should carefully choose (1) group size (i.e., thread-to-log sharing ratio) and (2) log buffer size, as they collectively determine both commit latency and the number of requests to object storage. Compared to per-thread logging, log

buffer size should be adjusted so that the average amount of log buffer space per worker thread is at least the log buffer size as in per-thread logging. This is necessary to guarantee meaningful reduction of flushes but also means using larger log buffers, while maintaining commit latency comparable to that of using per-thread logging. However, with larger log buffers (hence higher sharing ratio), their fill rate drops (i.e., it takes longer to fill out each log buffer until full). This leads to longer transaction commit latency under pipelined group commit which is necessary for achieving high throughput (Section 2.4). For example, suppose a transaction T has pre-committed as the first to insert to the log buffer in its thread group. T is then placed on the commit queue until the log buffer is full or a time out happens. In a busy system with constant activity, the former would be the usual case and T would be waiting on the commit queue until the log buffer is full and persisted with one S3 append request. The larger the log buffer is, the longer T needs to wait for other transactions to finish filling out the log buffer before T can be fully committed (i.e., removed from the commit queue).

Therefore, it is desirable to *proportionally* increase the log buffer size by group size. For example, with a group size of two (i.e., two threads sharing one log buffer), the log buffer size should be 2×.

Log Buffer Size vs. Transfer Latency. Increasing log buffer size proportional to group size mitigates the impact of lower log buffer fill rates, but can worsen log buffer flush delay since each request now is larger. Milliscale leverages the performance characteristics of S3 Express One Zone to solve this problem. As we have elaborated in Section 2.2, the latency to finish an append request only starts to grow linearly with request size beyond 1MB. Thus, it takes a similar amount of time to append a 512KB and a 1 MB log buffer. Consequently, setting the log buffer size to 1MB with a sharing ratio of two would lead to the commit latency equivalent to using per-thread logging with 512KB log buffers.

Figure 4 illustrates the tradeoff space of sharing ratio and log buffer size for throughput (left) and latency (right) under TPC-C (detailed setup in Section 5). Under per-thread logging (i.e., 1 worker per log buffer in the figure), throughput plateaus at 512KB log buffer. With larger log buffers (e.g., 1MB/2MB), a sharing ratio of two keeps the same throughput as per-thread logging because each thread on average has 512KB/1MB of log buffer space. Yet a sharing ratio that is too high (e.g., 4) would reduce the average log buffer space per thread, increase contention and flush frequency, eventually leading to lower throughput. Figure 4(right) shows the corresponding average commit latency. Different from Figure 4(left), a higher sharing ratio accelerates log buffer fill rate, leading to lower commit latency. Meanwhile, it is notable that under 1MB buffer size, a sharing ratio of two or four exhibits similar latency profile to using 512KB log buffers without any sharing. With dual-goal of maintaining high throughput (same as using per-thread logging or 1 worker per log buffer in the figure) and low commit latency, we therefore choose a sharing ratio of two and 1MB log buffer size. As our evaluation in Section 5 shows, such choice of sharing ratio and log buffer size is applicable to wide range of OLTP workload patterns, and can significantly reduce tail latency.

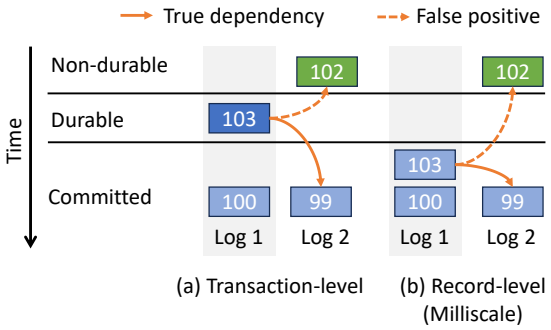

Figure 5: Transaction (a) vs. record (b) level dependency tracking which. Tracking at the record-level allows transaction with CSN 103 to commit, by skipping waiting for the transaction with CSN 102 to commit first.

4.3 Record-Level Dependency Tracking

Restricted decentralized logging reduces log flushes and log data volume, but still leaves additional software-induced delays compared to centralized logging (Section 2.4). To commit a transaction T, one needs to ensure both T's own log records and any log records T depends on are both persisted. This requires tracking dependencies between transactions and only fully commit transactions (i.e., notify the application) that meet both requirements.

False Positive Dependencies. Recording detailed, accurate dependency information (e.g., using dependency graphs) requires both additional log space and CPU cycles, so most systems approximate dependencies among transactions. Earlier work [20, 38] uses LSNs and GSNs to establish the partial order between transactions. However, these approaches are page-based that will incur false positive dependencies for transactions that access different records (thus no dependency) on the same page. More recent approaches employ more fine-grained, transaction-level dependency tracking [1, 26] avoid such false positive dependencies. Specifically, a transaction can only commit when its begin timestamp is lower than the lowest durable timestamp across all logs. However, we observe that this still can lead to false positives that significantly affect commit latency. Figure 5(a) shows an example with two logs, each dedicated to a group of threads (or a single thread under per-thread logging). The transactions are represented in boxes with their begin timestamp (or commit timestamps, depending on the system's concurrency control protocol); for example, we refer to the transaction with timestamp 103 as $T103$. Here, the lowest non-durable timestamp across all logs is 102, allowing any transactions below it (i.e., $T99$ and $T100$) to commit. Meanwhile, $T103$ has pre-committed with its log records fully persisted. However, since it is serialized after T102 (e.g., by acquiring its timestamp slightly later using atomic fetch-add), even though $T103$ does not depend on $T102$, it still cannot commit before $T102$ does, leading a false positive dependency.

Record-Level Dependency Tracking. Milliscale takes a step further to alleviate this problem with more fine-grained record-level dependency tracking. During forward processing, each transaction maintains the commit timestamp of its most recent direct predecessor that it depends on (called maximum dependency sequence

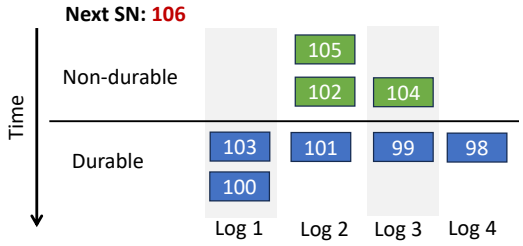

Figure 6: Milliscale takes the minimum non-durable CSN as the gCSN. If a log (Log 1 and Log 4) does not have a non-durable transaction, then its non-durable sequence number is considered as the next possible sequence number (106), i.e., gCSN = min(106, 102, 104, 106) = 102.

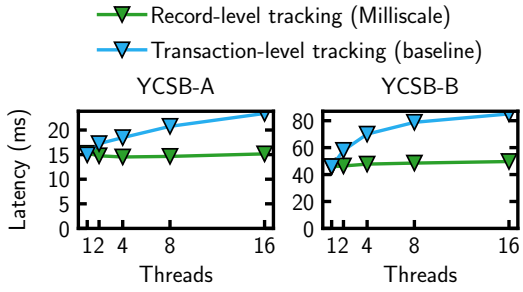

Figure 7: Average commit latency of YCSB-A (left) and YCSB-B (right) under uniform distribution. Milliscale avoids more false positive dependencies by tracking at the finer granularity of records than transaction-level tracking (baseline).

number, or DSN). For example, in Figure 5(b), if $T103$ reads or update a version produced by $T99$, it will update its DSN to be 99 if its current DSN is lower than 99. As log buffers get flushed, the engine computes a system-wide commit sequence number based on each log's durable and non-durable sequence numbers. Specifically, Milliscale will check all non-durable sequence numbers in each log and take the minimum as the commit sequence number, as shown in Figure 6. The pipelined group commit thread then can safely commit any transaction whose DSN is lower than the commit sequence number as this indicates that all the transaction's dependent transactions have committed. Compared to traditional transaction-level tracking, as shown in Figure 5(b), record-level tracking allows $T103$ to commit by avoiding a false positive dependency on $T103$.

Maintaining a DSN per transaction is lightweight. Since it is transaction-local, no additional concurrency control is required. Under MVCC (which Milliscale is based on), each version already carries the commit timestamp of its creator. The impact on recovery is also small, which we will discuss later. As shown in Figure 7, the average commit latency of YCSB-A (50%/50% read/update) and YCSB-B (95%/5% read/update) transactions under uniform distribution drops by up to over 35% when compared to transaction-level tracking. We further explore the effect of record-level dependency tracking on tail latency later in Section 5.

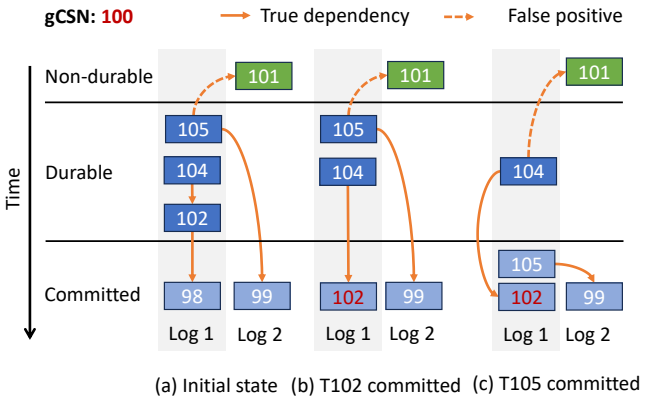

Figure 8: Pathological cases for record-level dependency tracking. (a) Initially, T102, T104 and T105 are durable and the global commit sequence number (gCSN) is 100. (b) T102 commits as its maximum dependent sequence number [DSN, 98] is smaller than gCSN. (c) T104 can not commit with a DSN [102] greater than gCSN, but T105 is allowed to commit since its latest dependent transaction [99] has already committed.

Analysis and Limitations. Record-level tracking reduces more false positives than prior approaches, but do not completely eliminate them. Figure 8 reasons about the merit and limitations of our approach when the system starts with an initial state where $T99$ and $T98$ have committed and the commit sequence number is 100. In Figure 8(a), since $T102$ only depends on $T98$, its DSN is 98, which is less than the commit sequence number (100). This means that all the dependencies of $T102$ have committed and since $T102$'s own log records are durable, $T102$ can safely commit. In Figure 8(b) $T104$ only depends on $T102$, which has committed. However, the committed sequence number is less than 102, which means that not all the transactions with a commit sequence number lower than 102 are persistent (e.g., $T101$). As a result, $T104$ can not commit and it remains to be explored how to eliminates such false positive dependencies completely with low overhead. As we show in later in Section 5, however, the impact of such pathological cases is negligible in a wide range of representative workloads.

4.4 Checkpointing and Recovery

As log objects accumulates, it is beneficial to reduce recovery time by checkpointing in-memory data to object storage. Milliscale inherits ERMIA's checkpointing approach [27] as the aforementioned restricted decentralized logging and record-level dependency tracking techniques do not affect checkpointing is performed. We describe how it works for completeness here. The main goal for an memory-optimized system like Milliscale and ERMIA is to store a consistent snapshot of the in-memory data. The checkpointing thread first obtains a transaction begin timestamp t by reading the central timestamp counter, and then scans the indirection array of each table to store the latest version that is committed before t. Thanks to MVCC, checkpointing does not block forward processing and a checkpoint will represent the state of the database as of

the beginning of the checkpoint. The log can then be truncated by deleting S3 objects that cover the checkpointed data.

The recovery protocol also remains similar to common redo-only logging designs [27, 30, 37–39] where multiple threads can proceed in parallel to scan the logs and race to install the latest versions. The only minor adaptation needed is to ensure transactions with a DSN greater than the global durable timestamp are preserved during recovery. For example, it was sufficient to replay the log until $T100$ in Figure 8 under transaction-level dependency tracking. However, under record-level dependency tracking Milliscale should ensure that transactions $T102$ and $T105$ survive after a crash. To facilitate this, we embed in each log record the transaction's DSN, and replay all log records whose dependent transactions have committed (i.e., with a commit timestamp greater than the log record's DSN).

Finally, most prior redo-only logging designs for main-memory systems store in each log record the entire new version, including fields that are not updated. This adds log data volume, leading to more S3 append requests. Milliscale stores only the delta (i.e., updated fields) in the log to reduce log data volume, but keep full versions in memory to avoid record reconstruction cost during forward processing. While this optimization is highly workload-dependent, as we show later in Section 5, storing only the delta can save as much as two-thirds of data volume in TPC-C.

4.5 Discussions

Restricted decentralized logging strikes a balance between contention avoidance (per-thread logging) and log buffer flush frequency. An alternative design is to still dedicate a private log buffer per thread, but allocate these log buffers in contiguous memory such that they can be flushed in one I/O. This would keep the full benefit of decentralized logging but require coordination among threads, e.g., to cope with cases where threads fill out their log buffers in different speeds which can delay the flush. We therefore opt for limited sharing within a group of threads. As mentioned in Section 4.2, log buffer size plays an important role and is set based on current latency characteristics of S3 Express One Zone; tuning it at runtime automatically is a promising future direction.

Milliscale can provide stronger availability than most alternatives. Compute-local SSDs are ephemeral, and so one must store data permanently in a disaggregated storage service that will survive compute restarts, such as EBS. However, its availability is limited to a single AZ. Although data in EBS can be periodically snapshot and stored in other AZs, the storage-level snapshot boundary may not coincide with that of the DBMS's, leaving the DBMS largely to handle replication. Moreover, Compared to S3 Standard, S3 Express One Zone provides the desirable low latency by trading off high availability guarantees. Therefore, systems like Milliscale may need to replicate data manually to reach the same level of high availability S3 Standard. Milliscale allows the application to provide a list of S3 buckets for replication. Upon group commit, we issue multiple asynchronous S3 append requests to all the buckets and start commit after all or a majority of requests have finished.

Finally, our focus in this paper has been adapting memory-optimized OLTP engines without buffer pools to use S3 Express One Zone. However, both techniques are applicable beyond such engines and can be leveraged by buffer pool-based systems [29, 32] since their logging subsystems largely follow the same designs. For example, one may apply restricted decentralized logging on top of the distributed logging design in LeanStore [29]. We leave such explorations to future work.

5 EVALUATION

We have shown the effect of restricted decentralized logging and record-level dependency tracking over a set of microbenchmarks earlier. Now we expand our evaluation of Milliscale with a wider range of baselines and benchmarks to demonstrate the following:

- Milliscale can easily outperform S3 standard in terms of latency and throughput across a variety of benchmarks.
- Milliscale can achieve comparable average and tail commit latency, if not lower, compared to baselines using block storage such as EBS while sustaining high throughput.
- Techniques in Milliscale are also proved useful for other storage backends, including on prem and cloud block storage.

5.1 Experimental Setup

Hardware. We perform experiments on a dedicated Amazon EC2 c6in.8xlarge instance over a variety of storage backends. The instance is equipped with an Intel Xeon Platinum 8375C CPU clocked at 2.9GHz (up to 3.5GHz with turboboost) and 64GB of memory. Although the CPU itself has 32 cores (64 hyperthreads), the instance exposes 32 vCPUs (hyperthreads) over 16 physical cores.

In addition to S3 Standard and S3 Express One Zone, we use EBS gp3 and io2 as baselines. Both gp3 and io2 are provisioned with 8,000 IOPS. The gp3 volume offers up to 2,000Mbps bandwidth while the io2 volume offers unlimited bandwidth. The instance supports 50Gbps of network bandwidth and 25Gbps of EBS bandwidth, and has no local NVMe SSD to lower overall cost of ownership, so all persistent data is either written to EBS or S3.

Software. The server runs Ubuntu 24.04.2 LTS and all code is compiled using GCC 13 with all optimizations enabled. We base our implementation on AWS SDK for C++ (version 1.11.676) [6]. The original AWS SDK provides asynchronous interfaces for S3 append, but does so by using a thread pool that is opaque to the application (i.e., Milliscale), which adds CPU scheduling overhead. We therefore implemented our own asynchronous S3 operations that internally spawns self-managed flusher threads to invoke the synchronous AWS S3 append interfaces.

Each worker thread group in Milliscale is allocated a flusher thread. We observe that they only take 3–5% of CPU cycles, so they could be pinned to the same hyperthread in budget constrained scenarios. To ease the interpretation of our results, we run experiments under 16 worker threads, each pinned to a different core. Each flusher thread is pinned to a different hyperthread of the same cores used by the corresponding worker thread group. For baselines that use block storage (gp3 or io2) and per-thread logs, we use io_uring to issue asynchronous I/O operations and each log (worker) has its own flusher thread which is necessary to keep up with the fill rate and sustain high throughput.

Experimental Variants. We implemented all variants on top of an improved version of ERMIA [27] by replacing its original centralized logging with per-thread decentralized logging with pipelined commit (Section 2.4), transaction-level dependency tracking and

delta-based logging (Section 4.4), and (2) using double buffering for log flush to overlap with forward processing, as commonly done in prior work. We then change the storage backend and commit protocol to build the following variants for our experiments:

- S3 Standard: Baseline that directly persists log records to S3 Standard, which enables record-level dependency tracking. It still keeps the per-thread dedicated logging for best average latency, because restricted decentralized logging does not improve tail latency atop storage backends as slow as S3 Standard.
- S3X: Baseline that directly persists log records to S3 Express One Zone, without optimizations proposed for Milliscale (i.e., naive use of S3 Express One Zone).
- gp3: Compared to S3X, it uses gp3 as the storage backend instead of S3 Express One Zone.
- io2: Compared to S3X, it uses io2 as the storage backend.
- Milliscale: Same as S3X but applies both restricted decentralized logging and record-level dependency tracking.

Same as prior work on OLTP engines [27, 29, 32, 33, 37], we implement Milliscale and baselines as shared libraries. The application (such as various benchmarks) directly uses the engine's C++ APIs to access database records, without SQL and optimizer layers. All the variants inherits the same concurrency control protocols in ERMIA and support read committed, snapshot isolation and serializability. The choice of isolation level is orthogonal to our evaluation, and we use snapshot isolation for all experiments.

Based on observations from Section 2.2, we use 512KB log buffer for variants that use per-thread logging. For Milliscale, we set log group size to two. Following analysis in Section 4.2, since each group consists of two threads, we set log buffer is 1MB (shared by two threads) so that on average each thread has the same space as in per-thread logging.

Benchmarks. We use both microbenchmarks based on YCSB [11] and end-to-end TPC-C benchmarks [36]. For YCSB, we use a database table of 30 million records. Each record is 80 bytes, including ten 8-byte fields, with one of them being the primary key. Each transaction follows a uniform or skewed (zipfian with theta=0.99) distribution to point read or update (read-modify-write) ten records through a Masstree [31] index built over the table's primary key column; the update operation reads the entire chosen record and modifies a random field in it. We test two transaction mix profiles, YCSB-A and YCSB-B, which respectively issues 50%/50% read/update and 95%/5% read/update transactions. For TPC-C, we use a database of 100 warehouses, and let each transaction randomly chooses a home warehouse to access. To focus on logging, we keep all table data in memory, so that all the traffic going to the storage backend is incurred by flushing log buffers.

We report the throughput, average commit latency and tail commit latency for all workloads. Note that for latency, we only measure the commit delay after forward processing. This allows us to fairly compare commit latency across all transaction types. Each experiment is run for 10 seconds and repeated for three times; the variance across runs is minuscule and so we report the average.

5.2 Microbenchmark Scalability

Our first set of experiments evaluate the throughput and commit latency under the YCSB-A and YCSB-B microbenchmarks.

Throughput. As shown in Figure 9(top), all variants except S3 Standard scale well as thread count increases under uniform distribution, thanks to pipelined commit and double buffering that are able to hide storage latency. The same trend is seen under the skewed distributed in Figure 9(top). S3 Standard's high latency well exceeds the workload's log buffer fill rate, forbidding it to scale. Compared to YCSB-A, YCSB-B includes more reads which leads to lower log buffer fill rate as a result of fewer updates (hence lighter log traffic). As a result, double buffering can hide most log flush delays and leading to higher throughput shown in Figure 10(top).

Average Latency. Next we explore how average commit latency changes under varying thread count; we explore tail latency later. Under the uniform distribution, Milliscale maintains nearly constant average latency (similar to single-threaded cases) across all thread counts as shown in Figure 9(bottom). Under 16 threads, Milliscale exhibits 28.4% lower average latency in YCSB-A than S3X; the number for YCSB-B in Figure 10(bottom) is 42% lower. These improvements are mainly a result of Milliscale's record-level dependency tracking which is more fine grained than prior approaches. Note that restricted decentralized logging can slightly increase latency due to more than one thread sharing the same log buffer. Compared to the default sharing ratio of 2:1, using a 1:1 ratio allows YCSB-A to achieves a 35% latency reduction under 16 threads. However, as we see later, restricted decentralized logging can more significantly reduce tail latency. YCSB-B benefits more from record-level dependency tracking: the workload itself features fewer writes and write transactions conflict less often under a uniform distribution, leading to fewer true dependencies that Milliscale can effectively handle.

Skewed workloads under the zipfian distribution present more dependencies across transactions, especially true dependencies. Yet record-level tracking is most effective for avoiding blocking transactions to commit due to false dependencies. Therefore, under YCSB-A in Figure 9(right) the average latency of Milliscale is close to that of S3X across all thread counts. YCSB-B in Figure 10(right) exhibits similar results, but with negligible gaps between S3X and EBS variants because the amount of writes is only 5%.

5.3 Tail Latency

In this experiment, we fix the number of threads to 16 and perform YCSB-A and YCSB-B workloads under both uniform and zipfian distributions. Figure 11 shows the commit latency for YCSB-A under different percentiles. Here, S3 Standard and io2 respectively exhibits the highest and lowest tail latency across all percentiles. They represent two extremes: cheap but slow vs. fast but expensive. Other variants attempt to strike a balance between these extremes. Milliscale and gp3 achieve similar tail latency profiles under uniform distribution, but the former starts to exhibit higher latency than gp3 under skewed accesses. We attribute the reason to more true dependencies under skewed workloads. Nevertheless, Milliscale still significantly reduces 99.9 percentile latency than S3X by 62.4%/51.8% under uniform/zipfian distributions. The numbers for 99.99 percentile tail latency are 56.2%/51.6%. Results of running YCSB-B as shown in Figure 12 give similar trends, so we do not repeat here.

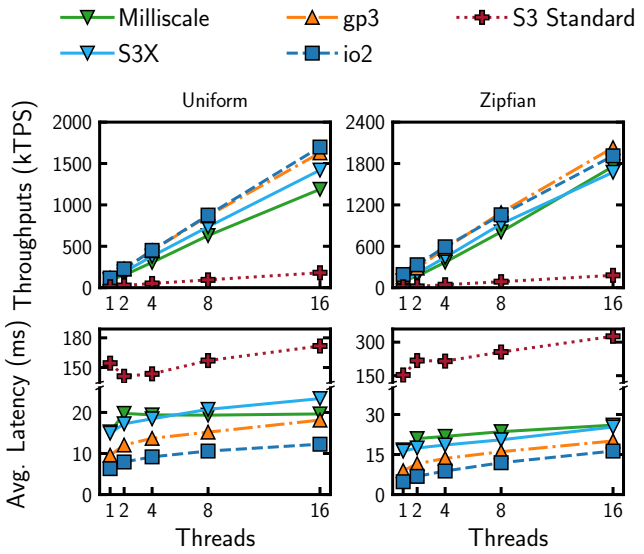

Figure 9: Throughput and average latency of YCSB-A (50% read, 50% write) under uniform (left) and zipfian (right) distributions. All but S3 Standard scale. Milliscale sustains high throughput as baselines and achieves latency similar to that of using block storage backends.

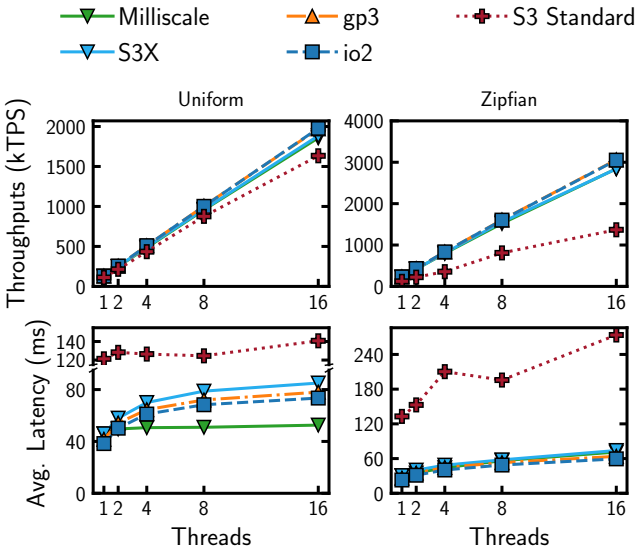

Figure 10: Throughput and average latency of YCSB-B (95% read, 5% update) under uniform (left) and zipfian (right) distributions. With more (in-memory) read operations Milliscale achieves lower latency than block storage, thanks to record-level dependency tracking.

5.4 Ablation Study

Section 4 has analyzed the effectiveness of restricted decentralized logging and record-level dependency tracking for average commit

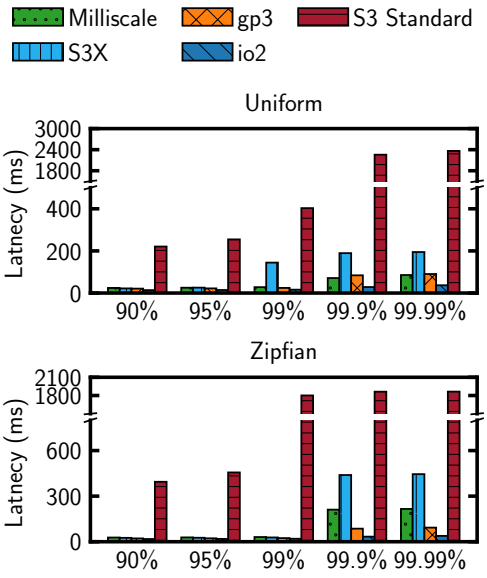

Figure 11: YCSB-A tail latency under 16 threads. Milliscale achieves comparable latency to gp3's under uniform distribution. Skewed workloads (bottom) present more true dependencies and increases latency for Milliscale, which however still exhibits over 50% lower latency at 99.9% and 99.99%.

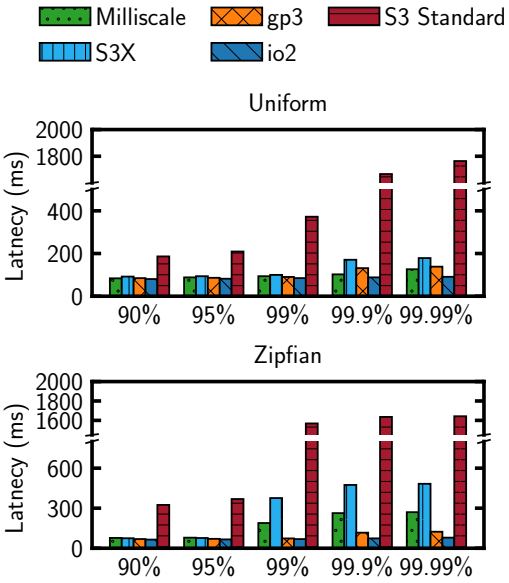

Figure 12: YCSB-B tail latency under 16 threads and uniform (top) and skewed (bottom) accesses.

latency. Now we expand the evaluation to tail latency using YCSB-A. We start with S3X, and then separately evaluate each technique under different percentiles. Figure 13 shows the result. All variants exhibit reasonably low latency until 95 percentile. As expected, restricted decentralized logging (S3X + RDL in the figure) exhibits

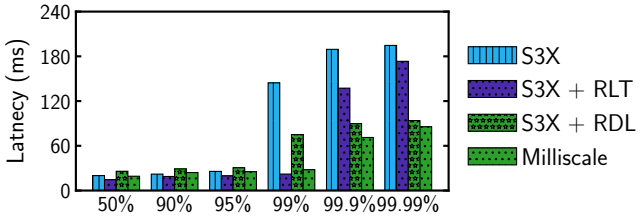

Figure 13: Effect of record-level dependency tracking (S3X + RLT), restricted decentralized logging (S3X + RDL) and their effects combined (Milliscale) under 16 threads and YCSB-A.

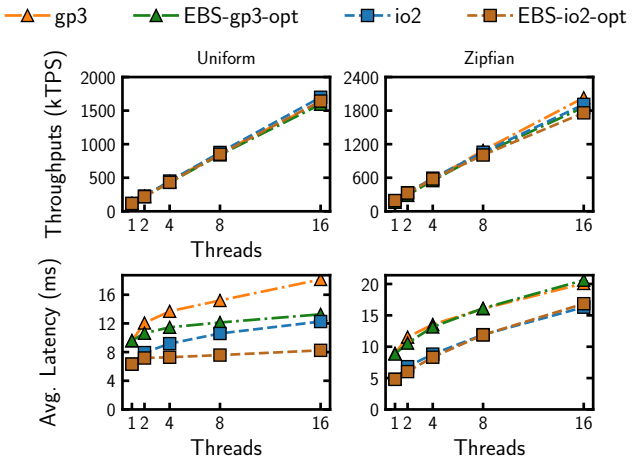

Figure 14: Throughput and average latency of YCSB-A when using EBS gp3/io2. Milliscale optimizations (*-opt) is mostly useful for under the uniform distribution for block storage.

slightly higher tail latency at 50 and 99 percentiles than record-level dependency tracking (S3X + RLT) due to slightly increased log buffer contention from threads in the same group that share a log buffer. Starting from 99 percentile, S3X's latency grows significantly by over 5×. Record-level dependency tracking helps stretch the trend to 99 percentile and remains helpful at 99.9 and 99.99 percentiles. At 99.9% and 99.99% restricted decentralized logging is more effective than record-level dependency tracking. We observe the reason is that with fewer log buffers, the workload inherently presents fewer cross-log dependencies that must be resolved before committing (e.g., $T105$ as identified in Figure 8).

5.5 Effect on Block Storage

As mentioned earlier, the usefulness techniques in Milliscale are not limited to object storage and can be applied to EBS or even on prem SSDs. We apply restricted decentralized logging (with the same 2:1 sharing ratio) and record-level dependency tracking on top of gp3 and io2. Figures 14 and 15 respectively show throughput/average latency and tail latency behaviors under YCSB-A. With optimizations from Milliscale, the average latency for gp3 and io2 under 16 threads decreased by 26.8% and 32.8%, respectively. The numbers for 99.99 percentile latency are 42.2% and 41.6%, separately. Similar to previous results, under the skewed zipfian distribution

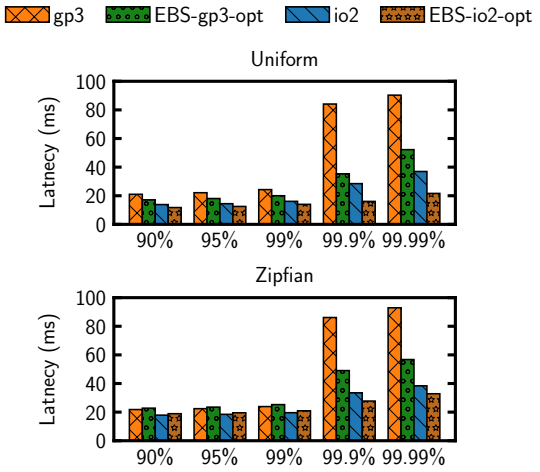

Figure 15: Tail latency of YCSB-A (16 threads) on block storage backends with vs. without Milliscale optimizations.

(where transactions exhibit more dependencies), average latency and throughput show no significant drop, and tail latency under 16 threads decreased by 39.0% for gp3 and 14.4% for io2.

Overall, these results indicate that Milliscale techniques are useful for faster storage backends, but are more needed and effective for higher latency counterparts, such as S3 Express One Zone.

5.6 End-to-End TPC-C Results

Our final experiment tests the performance of Milliscale using end-to-end TPC-C. As mentioned earlier, we use a database of 100 warehouses and let each worker thread randomly choose its home warehouse. This allows to stress the system with more cross-log dependencies and increases contention between transactions. With a 2:1 sharing ratio, restricted decentralized logging in Milliscale cuts the number of S3 append requests by ~50% across different scales in Figure 16. As show in Figure 17, Milliscale outperforms the baseline S3 Standard significantly in terms of both throughput and latency. Milliscale's average latency is 3.8% higher than S3X and 23% higher than gp3 under 16 threads. Under 99 percentile, both S3X and Milliscale exhibit much higher latency than gp3 and io2, which we attribute to the difference of each storage backend's performance characteristics: S3 Express One Zone's tail latency starts to spike at 99 percentile, while for gp3 and io2 this starts at 99.9 percentile. Nevertheless, as shown in Figure 18, similar to earlier microbenchmark results, Milliscale effectively lowers latency compared to S3X and achieves similar to gp3's latency at 99.9 and 99.99 percentiles.

6 RELATED WORK

Our work is most related to prior work that leverages object storage for DBMSs and optimizes write-ahead logging.

DBMSs over Object Storage. The earliest attempt (to the best of our knowledge) of using S3 for DBMS was done by Brantner et al. [10]. The system was based on S3 Standard and had to deal with many drawbacks of S3, in particular weak consistency, lack of append support and very high latency (100ms-level). These led

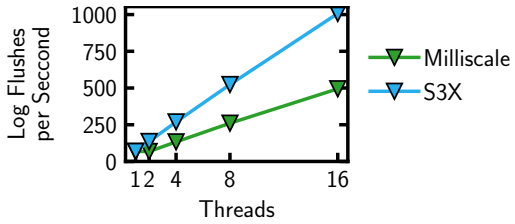

Figure 16: Number of S3 append requests per second under TPC-C using restricted decentralized logging with a 2:1 sharing ratio (1MB log buffer shared by two threads) vs. per-thread logging (512KB buffer per thread).

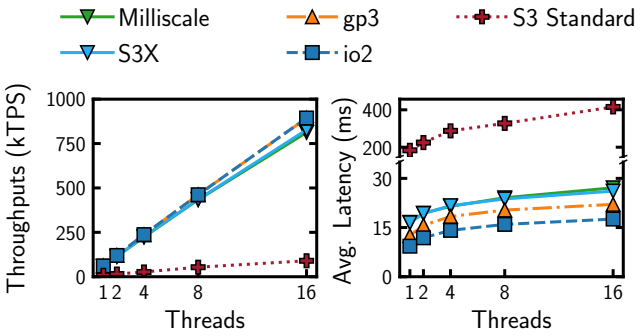

Figure 17: TPC-C throughput and average latency.

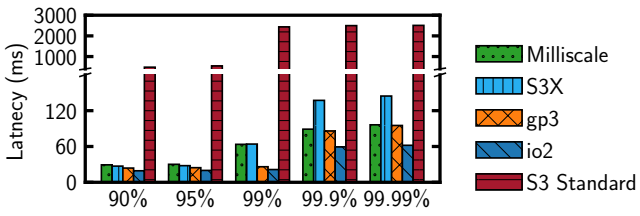

Figure 18: TPC-C tail latency under 16 threads.

to solutions that are not fully ACID compliant and logging was offloaded to a separate message queue implementation. The high end-to-end transaction latency (seconds-level) has discouraged these approaches. As Section 2 describes, these issues are not largely resolved (strong consistency and append support) or mitigated (latency) by S3 Express One Zone, which allowed us to build Milliscale. Delta Lake [8] manages transactions over S3 Standard for data lake applications. To bridge the speed gap between memory and S3 Standard, it relies on heavy caching using compute-side SSDs. Other approaches in this space mostly focused on OLAP or hybrid workloads over S3 Standard. Colibri [35] proposes a hybrid architecture for OLTP and OLAP workloads to leverage cloud storage and separate hot and cold pages. AnyBlob [15] presents an io_uring-based download manager that asynchronously handles concurrent S3 GET requests, enabling DBMSs to saturate fast data center networks for read-heavy OLAP workloads while minimizing CPU overhead. LiquidCache [19] deploys a cache server between compute nodes and object storage. While data is stored in object storage as Parquet files, the cache server uses a custom in-memory format optimized for fast filter evaluation. PushdownDB [43] uses the (now end-of-life) S3 Select [16] feature to push filter, projection, and aggregation operations directly to S3 storage nodes, reducing network traffic between storage and compute for OLAP workloads. FlexPushdownDB [42] extends PushdownDB with a hybrid architecture that combines caching with computation pushdown at fine granularity. Compared to prior work, Milliscale targets a different storage backend (S3 Express One Zone) and focuses on optimizing write-ahead logging for lower commit latency.

Logging Optimizations. Milliscale is built on top of several important prior ideas for scalable logging. Aether [23] evaluated and showed the potential of early lock release and log flush pipelining, which allowed subsequent systems to improve transaction throughput to the same level of asynchronous commit, yet without sacrificing correctness. However, the inherent centralized logging bottleneck still demands decentralized logging [24, 41, 45], which later became a standard design decision in memory-optimized OLTP engines [29, 30, 32, 37]. A major drawback in traditional decentralized logging is that it trades transaction commit latency for throughput. Cross-log dependencies are a major source, and various prior approaches have been proposed to mitigate their impact. For example, LeanStore uses remote flush avoidance [20] to skip unnecessary log buffer flushes if a transaction does not depend on other logs. Autonomous commit [33] parallelizes small log flushes across worker threads and proactively inserts dummy transactions to advance GSNs to mitigate the impact of stragglers. Border-Collie [26] proposes wait-free algorithms to efficiently find the upper bound logical clock position for transactions to safely commit. ELEDA [25] fill gaps in LSNs across logs so that the globally durable LSN marker can be advanced timely. Compared to these approaches, Milliscale tracks and resolves dependencies at the finer record granularity while maintaining low overhead.

7 SUMMARY

We have explored the feasibility of leveraging recent low-latency, mutable object storage services as represented by Amazon S3 Express One Zone for OLTP write-ahead logging. While traditional decentralized logging approaches can deliver high throughput, they present high transaction commit latency (especially tail latency) over S3 Express One Zone. Based on S3 Express One Zone's performance characteristics, we propose Milliscale, a memory-optimized OLTP engine that uses S3 Express One Zone as write-ahead logging storage to optimize commit latency. Milliscale consists of two key techniques—restricted decentralized logging and record-level dependency tracking—that respectively reduces log flush frequency without imposing higher log data transfer time and avoids false positive dependencies commonly found in prior work. These techniques can also be applied in other systems or storage backends. Evaluation using both microbenchmarks and end-to-end TPC-C showed that Milliscale can effectively reduce both average and tail latency, while sustaining high throughput.

REFERENCES

[1] Adnan Alhomssi, Michael Haubenschild, and Viktor Leis. 2023. The evolution of leanstore. In *BTW 2023*. Gesellschaft für Informatik eV, 259–281.

[2] Amazon Web Services. 2025. Amazon EBS Volume Types. https://aws.amazon.com/ebs/volume-types/

[3] Amazon Web Services. 2025. Amazon S3 Express One Zone storage class. https://aws.amazon.com/s3/storage-classes/express-one-zone/

[4] Amazon Web Services. 2025. Amazon S3 Pricing. https://aws.amazon.com/s3/pricing/ Accessed: 2025-10-24.

[5] Amazon Web Services. 2025. Amazon S3 Storage Classes. https://aws.amazon.com/s3/storage-classes/

[6] Amazon Web Services. 2025. AWS SDK for C++ version 1.11.676. https://github.com/aws/aws-sdk-cpp/releases/tag/1.11.676

[7] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, et al. 2019. Socrates: The New SQL Server in the Cloud. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, 1743–1756.

[8] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Łuszczak, Michał undefinedwitakowski, Michał Szafrański, Xiao Li, Takuya Ueshin, Mostafa Mokhtar, Peter Boncz, Ali Ghodsi, Sameer Paranjpye, Pieter Senster, Reynold Xin, and Matei Zaharia. 2020. Delta lake: high-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3411–3424.

[9] AWS. 2026. Attach an EBS volume to multiple EC2 instances using Multi-Attach. https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volumes-multi.html

[10] Matthias Brantner, Daniela Florescu, David Graf, Donald Kossmann, and Tim Kraska. 2008. Building a database on S3. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. 251–264.

[11] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.

[12] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16)*. 215–226.

[13] Alex Depoutovitch, Chong Chen, Jin Chen, Paul Larson, Shu Lin, Jack Ng, Wenlin Cui, Qiang Liu, Wei Huang, Yong Xiao, and Yongjun He. 2020. Taurus Database: How to Be Fast, Available, and Frugal in the Cloud. (2020), 1463–1478.

[14] DuckDB Documentation. 2025. S3 API Support. https://duckdb.org/docs/stable/core_extensions/httpfs/s3api

[15] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proc. VLDB Endow.* 16, 11 (July 2023), 2769–2782.

[16] Arushi Garg and Preethi Raajaratnam. 2024. How to optimize querying your data in Amazon S3. https://aws.amazon.com/blogs/storage/how-to-optimize-querying-your-data-in-amazon-s3

[17] Google. 2025. Google Cloud Documentation – Storage Classes. https://docs.cloud.google.com/storage/docs/storage-classes?_gl=1*18hxa54*_up*MQ..&gclid=Cj0KCQiAo4TKBhDRARIsAGW29beKlmBi5jG3PCEP_0qzQMj8FiGT3vSd0LO5lRy5-IyTrRk9HiRgqx0aAt_cEALw_wcB&gclsrc=aw.ds#standard

[18] Gabriel Haas and Viktor Leis. 2023. What Modern NVMe Storage Can Do, and How to Exploit It: High-Performance I/O for High-Performance Storage Engines. *Proc. VLDB Endow.* 16, 9 (may 2023), 2090–2102.

[19] Xiangpeng Hao, Andrew Lamb, Yibo Wu, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2025. LiquidCache: Efficient Pushdown Caching for Cloud-Native Data Analytics. *Proc. VLDB Endow.* 16 (2025).

[20] Michael Haubenschild, Caetano Sauer, Thomas Neumann, and Viktor Leis. 2020. Rethinking Logging, Checkpoints, and Recovery for High-Performance Storage Engines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. 877–892.

[21] IEEE and The Open Group. 2016. The Open Group Base Specifications Issue 7, IEEE Std 1003.1. (2016).

[22] Intel Corporation. 2016. Intel 64 and IA-32 Architectures Software Developer Manuals. (Oct. 2016).

[23] Ryan Johnson, Ippokratis Pandis, Radu Stoica, Manos Athanassoulis, and Anastasia Ailamaki. 2010. Aether: A Scalable Approach to Logging. *Proc. VLDB Endow.* 3, 1 (Sept. 2010), 681–692.

[24] Ryan Johnson, Ippokratis Pandis, Radu Stoica, Manos Athanassoulis, and Anastasia Ailamaki. 2012. Scalability of write-ahead logging on multicore and multi-socket hardware. *The VLDB Journal* 21, 2 (April 2012), 239–263.

[25] Hyungsoo Jung, Hyuck Han, and Sooyong Kang. 2017. Scalable database logging for multicores. *Proc. VLDB Endow.* 11, 2 (Oct. 2017), 135–148.

[26] Jongbin Kim, Hyeongwon Jang, Seohui Son, Hyuck Han, Sooyong Kang, and Hyungsoo Jung. 2019. Border-Collie: A Wait-free, Read-optimal Algorithm for Database Logging on Multicore Hardware. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. 723–740.

[27] Kangnyeon Kim, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. ERMIA: Fast memory-optimized database system for heterogeneous workloads. In *Proceedings of the 2016 International Conference on Management of Data*. 1675–1687.

[28] Andrew Lamb, Yijie Shen, Daniël Heres, Jayjeet Chakraborty, Mehmet Ozan Kabak, Liang-Chi Hsieh, and Chao Sun. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD '24)*. 5–17. https://doi.org/10.1145/3626246.3653368

[29] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. 2018. LeanStore: In-Memory Data Management beyond Main Memory. In *2018 IEEE 34th International Conference on Data Engineering (ICDE) (IEEE ICDE)*. 185–196.

[30] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. 2017. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*. 21–35.

[31] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache craftiness for fast multicore key-value storage. In *Proceedings of the 7th ACM european conference on Computer Systems*. 183–196.

[32] Thomas Neumann and Michael J Freitag. 2020. Umbra: A Disk-Based System with In-Memory Performance. In *10th Conference on Innovative Data Systems Research, CIDR 2020, Amsterdam, The Netherlands, January 12-15, 2020, Online Proceedings*.

[33] Lam-Duy Nguyen, Adnan Alhomssi, Tobias Ziegler, and Viktor Leis. 2025. Moving on From Group Commit: Autonomous Commit Enables High Throughput and Low Latency on NVMe SSDs. *Proc. ACM Manag. Data* 3, 3, Article 191 (June 2025), 24 pages.

[34] Raghu Ramakrishnan and Johannes Gehrke. 2003. *Database Management Systems* (3 ed.).

[35] Tobias Schmidt, Dominik Durner, Viktor Leis, and Thomas Neumann. 2024. Two Birds With One Stone: Designing a Hybrid Cloud Storage Engine for HTAP. *Proc. VLDB Endow.* 17, 11 (July 2024), 3290–3303. https://doi.org/10.14778/3681954.3682001

[36] TPC. 2010. TPC Benchmark C (OLTP) Standard Specification, revision 5.11. http://www.tpc.org/tpcc

[37] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy Transactions in Multicore In-Memory Databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)*. 18–32.

[38] Tianzheng Wang and Ryan Johnson. 2014. Scalable Logging through Emerging Non-Volatile Memory. *Proc. VLDB Endow.* 7, 10 (June 2014), 865–876.

[39] Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2017. Query Fresh: Log Shipping on Steroids. *Proc. VLDB Endow.* 11, 4 (Dec. 2017), 406–419.

[40] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proc. VLDB Endow.* 10, 7 (March 2017), 781–792.

[41] Yu Xia, Xiangyao Yu, Andrew Pavlo, and Srinivas Devadas. 2020. Taurus: Lightweight Parallel Logging for in-Memory Database Management Systems. *Proc. VLDB Endow.* 14, 2 (Oct. 2020), 189–201.

[42] Yifei Yang, Matt Youill, Matthew Woicik, Yizhou Liu, Xiangyao Yu, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. 2021. FlexPushdownDB: hybrid pushdown and caching in a cloud DBMS. *Proc. VLDB Endow.* 14, 11 (July 2021), 2101–2113.

[43] Xiangyao Yu, Matt Youill, Matthew Woicik, Abdurrahman Ghanem, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. 2020. PushdownDB: Accelerating a DBMS Using S3 Computation. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1802–1805.

[44] Matei Zaharia. 2025. Bringing the Operational and Analytical Worlds Together with Lakebase. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5539.

[45] Wenting Zheng, Stephen Tu, Eddie Kohler, and Barbara Liskov. 2014. Fast databases with fast durability and recovery through multicore parallelism. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation* (Broomfield, CO) *(OSDI'14)*. 465–477.